TEACHER ANSWER KEY · DO NOT PRINT FOR STUDENTS

Quiz: adding an item to a stack, a queue and a linear linked list

20 minutes · 20 marks · start of the next lesson, before pop, dequeue and delete

New numbers on purpose: not 15, 28, 41 (lesson) and not 7, 9, 4 (exit strip). Same scope as the lesson: push, enqueue, one insert, and the overflow test. No pop, dequeue, delete or circular queue.

Curriculum link

Item	ICT C&A Guide (S4–6) wording it practises
All	Elective C (a)(i): “Construct lists, stacks and queues in terms of arrays”
B1	Compulsory D(b): “Perform a dry run of a set of steps to determine its … output”
B2	Compulsory D(b): “Produce a trace table to show values of variables at each stage”
B3	Compulsory D(b): “Locate logic error in an algorithm and correct it”

Section A · Answers

Q	Ans	Why / what each wrong option shows	Obj.
A1	A	B overwrites S[3] without moving TOP. C writes at the left end. D moves TOP the wrong way.	2
A2	A	Enqueue never assigns FRONT. HEAD changes only for an insert at the start, not in the middle. Matches the pointer card.	5
A3	B	A changes FRONT during enqueue. C overwrites Q[2]. D writes at the front.	3
A4	D	Follow NEXT from HEAD: node 1 → 3 → 2. A reads the array in index order.	4
A5	C	A and B make node 4 point to itself. D copies null into NEXT[1] and cuts the list.	4
A6	B	(3) is the array idea from the lesson opening. In a linked list the later values stay put.	1, 5

Section B · Programs and traces (14 marks)

Blue cells are given. Write every yellow cell. Leave a cell blank if it stays empty.

B1. Stack · dry run a program (4 marks)

A stack is stored in S[1..6]. Before the program runs, the stack holds four items and TOP = 4, as shown on the right.

Program

```
1   for I ← 1 to 3
2      X ← I * 10
3      if TOP = 6 then
4         output “stack overflow”
5      else
6         TOP ← TOP + 1
7         S[TOP] ← X
8      endif
9   next I
10  output TOP
```

Stack S (top of the drawing = S[6])

Index	S
6	20
5	10
4	5
3	17
2	3
1	8

TOP = 6

(a) Complete the stack on the right after the program ends, and write the final value of TOP. [2]

(b) Write everything the program outputs, in order. [1]	`stack overflow, then 6`

(c) One value of X is never stored. Which value, and why? [1]

`30. When I = 3, TOP is already 6, so line 4 runs instead of lines 6–7.`

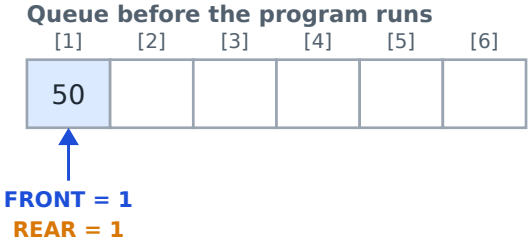
B2. Queue · trace table (3 marks)

A queue is stored in Q[1..6]. Before the program runs, Q[1] = 50, FRONT = 1 and REAR = 1. The array IN holds IN[1] = 62, IN[2] = 73 and IN[3] = 81.

Program

```
1   for I ← 1 to 3
2      if REAR = 6 then
3         output “queue overflow”
4      else
5         REAR ← REAR + 1
6         Q[REAR] ← IN[I]
7      endif
8   next I
9   output Q[FRONT], Q[REAR]
```

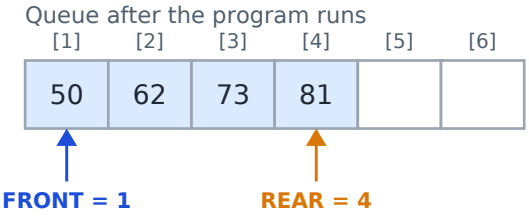
Queue before the program runs



(a) Complete the trace table. Write the values **at the end of** each pass of the loop. [2]

I	IN[I]	FRONT	REAR	Cell written
1	62	1	2	Q[2]
2	73	1	3	Q[3]
3	81	1	4	Q[4]

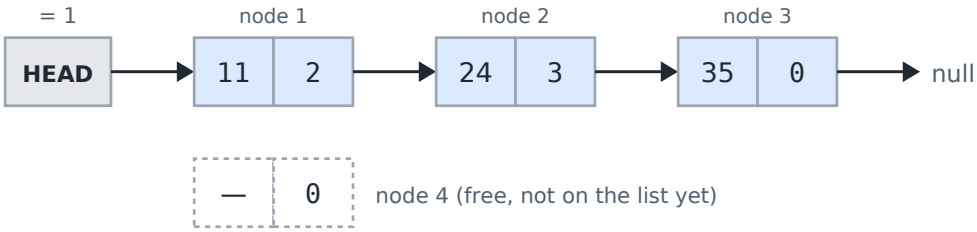
(b) Write the output of line 9. [1]	`50, 81`



B3. Linear linked list · find and correct the logic error (7 marks)

HEAD = 1. The list is 11 → 24 → 35 → null. Node 4 is free. The aim is to insert **30 between 24 and 35**.

The list before the insert



Each node: DATA | NEXT. NEXT = 0 means null.

Arrays before the insert

Index	DATA	NEXT
1	11	2
2	24	3
3	35	0
4	—	0

A student wrote this program to do the insert

```
1  DATA[4] ← 30
2  NEXT[2] ← 4
3  NEXT[4] ← NEXT[2]
```

(a) Dry run the student's program. After it ends, what are NEXT[2] and NEXT[4]? [2]

NEXT[2] =	4	NEXT[4] =	4

(b) Which value can no longer be reached from HEAD, and why? [1]

35. Line 2 overwrites NEXT[2] before it is copied, so NEXT[4] copies 4 and node 4 points to itself. The arrow from 24 to 35 is lost.

(c) Rewrite the program so the insert works. [2]

```
1  DATA[4] ← 30
2  NEXT[4] ← NEXT[2]
3  NEXT[2] ← 4
```

(d) Complete the arrays after **your** program in (c) runs. [2]

Index	DATA	NEXT
1	11	2
2	24	4
3	35	0
4	30	3

The list after the correct insert



Each node: DATA | NEXT. NEXT = 0 means null.

Marking notes

- B1(a): 1 mark for S[5] = 10 and S[6] = 20 with S[1..4] unchanged; 1 mark for TOP = 6. Writing 30 anywhere means the overflow test was missed.
- B1(b): both outputs, in order, for the mark.
- B2(a): 2 marks if all yellow cells are right; 1 mark with one wrong row. A FRONT column that changes is the lesson's FRONT error.
- B2(b): 50, 81 for the mark. A student who writes 62 first has moved FRONT.
- B3(a): 1 mark each. NEXT[4] = 3 means the student ran the lines in the safe order, not as written.
- B3(b): accept any wording naming 35 and the overwritten arrow.
- B3(c): 1 mark for NEXT[4] ← NEXT[2] before NEXT[2] ← 4; 1 mark for all three lines correct. Accept DATA[4] ← 30 in any position.
- B3(d): 1 mark for node 4 = (30, 3); 1 mark for NEXT[2] = 4 with nodes 1 and 3 unchanged.
- Objective map: 1 → A6; 2 → A1, B1; 3 → A3, B2; 4 → A4, A5, B3; 5 → A2, A6.
- Errors to look for, as in the lesson plan: value written at the left end; FRONT changed during enqueue; the old NEXT overwritten before it is copied.